

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

FINAL YEAR PROJECT REPORT

AI Bike Blind Spot Detection: Intelligent Blind Spot Monitoring System



GOVT.GRADUATE COLLEGE (B) GULBERG, LAHORE

Submitted By:	Muhammad Mohsin (Roll No: 061118)
Supervisor:	Prof. Hafiz Ahmad Shahzad
Session:	2021–2026
Semester:	8th Semester
Year:	2026

Bachelor of Science in Information Technology (2021–2025)
Department of Computer Science, Govt. Graduate College (B) Gulberg, Lahore, Pakistan

Acknowledgment

All praise and glory be to Allah Almighty, the Most Gracious and the Most Merciful, for bestowing upon me the knowledge, patience, and perseverance to complete this project successfully.

I extend my deepest gratitude to my project supervisor, Prof. Hafiz Ahmad Shahzad, whose invaluable guidance, constructive feedback, and unwavering encouragement throughout the development of this project have been indispensable. His technical expertise and mentorship shaped the direction and quality of this work significantly.

I am also grateful to the faculty and staff of the Department of Computer Science at Govt. Graduate College (B) Gulberg, Lahore, for providing the academic environment and resources that made this work possible.

Special appreciation goes to my family for their continuous moral support, patience, and belief in me throughout the duration of my studies. Finally, I acknowledge the open-source communities behind Python, Django, scikit-learn, PyTorch, ONNX Runtime, and Chart.js, whose tools formed the technological backbone of this project.

Muhammad Mohsin

Roll No: 061118

BSIT 8th Semester — 2026

APPROVAL CERTIFICATE

This is to certify that the Final Year Project titled:

“AI bike blind spot detection: Intelligent Bike Riding Assistant”

has been completed by Muhammad Mohsin (Roll No: 061118) in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology from the Department of Computer Science, Govt. Graduate College (B) Gulberg, Lahore, Pakistan.

The project has been evaluated and approved by the undersigned.

Prof. Hafiz Ahmad Shahzad

Project Supervisor
Dept. of Computer Science

Head of Department

Department of Computer Science
Govt. Graduate College (B) Gulberg

Date: _____

Stamp: _____

Abstract

Road safety remains a critical public health challenge in Pakistan and across South Asia, where motorcycle accidents account for a disproportionate share of traffic fatalities. The majority of these accidents stem from preventable causes: excessive speeding, abrupt lane changes, hard braking, and inadequate situational awareness. Existing solutions are either too expensive, require proprietary hardware, or lack real-time intelligence.

This report presents AI bike blind spot detection: an Intelligent Blind Spot Monitoring System — a low-cost, software-defined safety system that fuses GPS and accelerometer sensor data through a Kalman filter, classifies riding events using a trained machine learning model, and delivers sub-50 millisecond safety alerts to the rider. The system is deployed as a Django REST Framework (DRF) backend serving a plain HTML + Chart.js evaluation dashboard, requiring only an Android smartphone mounted on the handlebar.

The sensor pipeline operates at 10 Hz (GPS) and 50 Hz (accelerometer), producing a 7-feature vector per frame. A Gradient Boosting classifier (scikit-learn) was trained to classify five event types: normal riding, hard braking, sharp turning, pothole, and overspeeding. A continuous risk score (0.0–1.0) is also computed per frame; scores above 0.7 trigger a HIGH_RISK alert. The model was exported to ONNX format for framework-agnostic inference and achieves a recall of greater than 0.92 on high-risk events with inference latency below 30 ms at the 95th percentile.

The system was validated over 30 or more distinct ride sessions covering urban, highway, and mixed conditions. End-to-end pipeline latency was confirmed below 50 ms on the 95th percentile. All experiments were tracked using MLflow. The evaluation dashboard provides live visualisation of speed, risk score, alert history, and model performance metrics directly from the DRF API.

Keywords: *Bike Safety, GPS, Accelerometer, Kalman Filter, Gradient Boosting, ONNX, Django REST Framework, Real-Time Inference, Collision Detection*

Table of Contents

Approval Certificate	ii
Acknowledgment	iii
Abstract	iv
List of Figures	vii
List of Tables	viii
Chapter 1: Introduction	1
1.1 Background of the Project	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope of the Project	3
1.5 Significance of the Project	4
Chapter 2: Literature Review	5
2.1 Existing Systems and Research	5
2.2 Comparative Analysis	6
2.3 Research Gap	7
2.4 Proposed Solution	8
Chapter 3: System Analysis and Design	9
3.1 Requirement Analysis	9
3.2 Functional Requirements	9
3.3 Non-Functional Requirements	10
3.4 Use Case Diagram	11
3.5 Activity Diagram	11
3.6 Class Diagram	12
3.7 System Architecture	12
3.8 Database Design (ERD)	13
Chapter 4: Implementation	15
4.1 Development Methodology	15
4.2 Technologies and Tools Used	15
4.3 System Modules	16
4.4 Database Implementation	17
4.5 User Interface Screenshots	18
4.6 Code Structure Overview	18
Chapter 5: Testing and Results	20
5.1 Testing Methodology	20
5.2 Test Cases	20
5.3 Results and Analysis	22
5.4 Performance Evaluation	23
5.5 Discussion	24

Chapter 6: Conclusion and Future Work25

6.1 Conclusion25

6.2 Achievements25

6.3 Limitations26

6.4 Future Enhancements26

References27

Appendix A: Source Code Snippets29

Appendix B: User Manual31

List of Figures

Figure 3.1 — Five-Layer System Pipeline Diagram12

Figure 3.2 — Use Case Diagram — AI bike blind spot detection11

Figure 3.3 — Activity Diagram — Sensor Frame Processing12

Figure 3.4 — Class Diagram — DRF Backend Models13

Figure 3.5 — Entity Relationship Diagram (ERD)14

Figure 4.1 — Agile Sprint Schedule15

Figure 4.2 — Evaluation Dashboard — Speed Gauge View18

Figure 4.3 — Evaluation Dashboard — Risk Timeline View19

Figure 5.1 — Confusion Matrix — Gradient Boosting Classifier23

Figure 5.2 — ROC-AUC Curves per Event Class23

Figure 5.3 — Inference Latency Histogram (p95 < 30 ms)24

List of Tables

Table 2.1 — Comparative Analysis of Existing Systems6

Table 3.1 — Functional Requirements9

Table 3.2 — Non-Functional Requirements10

Table 3.3 — Database Schema — RideSession13

Table 3.4 — Database Schema — SensorFrame13

Table 3.5 — Database Schema — InferenceResult14

Table 3.6 — Database Schema — AlertEvent14

Table 4.1 — Justified Technology Stack16

Table 4.2 — Deliberately Excluded Components17

Table 4.3 — Ground-Truth Label Schema17

Table 5.1 — Unit Test Cases21

Table 5.2 — Integration Test Cases21

Table 5.3 — Evaluation Metrics Results22

Table 5.4 — Sensor Alert Thresholds23

Chapter 1: Introduction

1.1 Background of the Project

Road traffic injuries are a leading cause of death globally, with motorcycles accounting for a disproportionate share of fatalities in developing countries. In Pakistan, motorcycles are the primary mode of transport for millions of urban and rural commuters. According to traffic safety reports, the majority of motorcycle fatalities involve at least one of the following factors: overspeeding, abrupt lane changes, poor road surface awareness, or failure to react in time to a developing hazard.

Despite rapid advances in automotive safety technology — including adaptive cruise control, lane departure warnings, and automatic emergency braking — these systems are confined to high-end vehicles and are entirely absent from the motorcycle segment. The economic gap between available technology and typical motorcycle riders in South Asia creates a critical safety deficit that this project aims to address.

Artificial intelligence, machine learning, and low-cost inertial sensors have reached a maturity level that makes it feasible to build a safety-aware riding assistant using only a consumer-grade Android smartphone. The AI bike blind spot detection project was conceived to close this gap: to deliver real-time, sensor-fused safety intelligence to motorcycle riders without requiring specialised or expensive hardware.

1.2 Problem Statement

The majority of motorcycle accidents in Pakistan and similar markets are caused by three interrelated problems: (1) riders lack real-time feedback on whether their speed is appropriate for current road and traffic conditions; (2) dangerous riding events such as hard braking and sharp cornering are not detected and recorded in any actionable form; and (3) no affordable, software-defined system exists that can classify riding risk in real time and alert the rider before a collision occurs.

Existing commercial solutions such as dashcams and GPS trackers record data passively for post-incident review but provide no real-time alert. Academic proposals in this space typically rely on proprietary hardware, cloud latency, or architectures that are

not reproducible within an FYP scope. The result is a gap between raw sensor capability — widely available in every Android smartphone — and actionable, sub-50-millisecond safety feedback.

Core Problem: How can GPS and accelerometer data from a low-cost Android smartphone be fused, classified, and converted into a real-time rider alert within a 50-millisecond end-to-end latency budget?

1.3 Objectives

The primary objectives of this project are:

1. Design and implement a five-layer sensor-fusion pipeline that processes GPS and accelerometer data in real time with a total end-to-end latency of less than 50 milliseconds.
2. Collect and label a riding dataset of 30 or more distinct sessions covering urban, highway, and mixed riding conditions across five event classes: normal, hard_brake, sharp_turn, pothole, and overspeeding.
3. Train and evaluate a Gradient Boosting classifier achieving recall greater than 0.92 on high-risk events and ROC-AUC greater than 0.95, with all experiments tracked in MLflow.
4. Export the trained model to ONNX format and integrate it into a Django REST Framework backend API with a documented and tested endpoint contract.
5. Build and validate a real-time evaluation dashboard using plain HTML and Chart.js that streams live inference results from the DRF API.
6. Document all failure modes, security weaknesses, and system limitations explicitly for responsible academic submission.

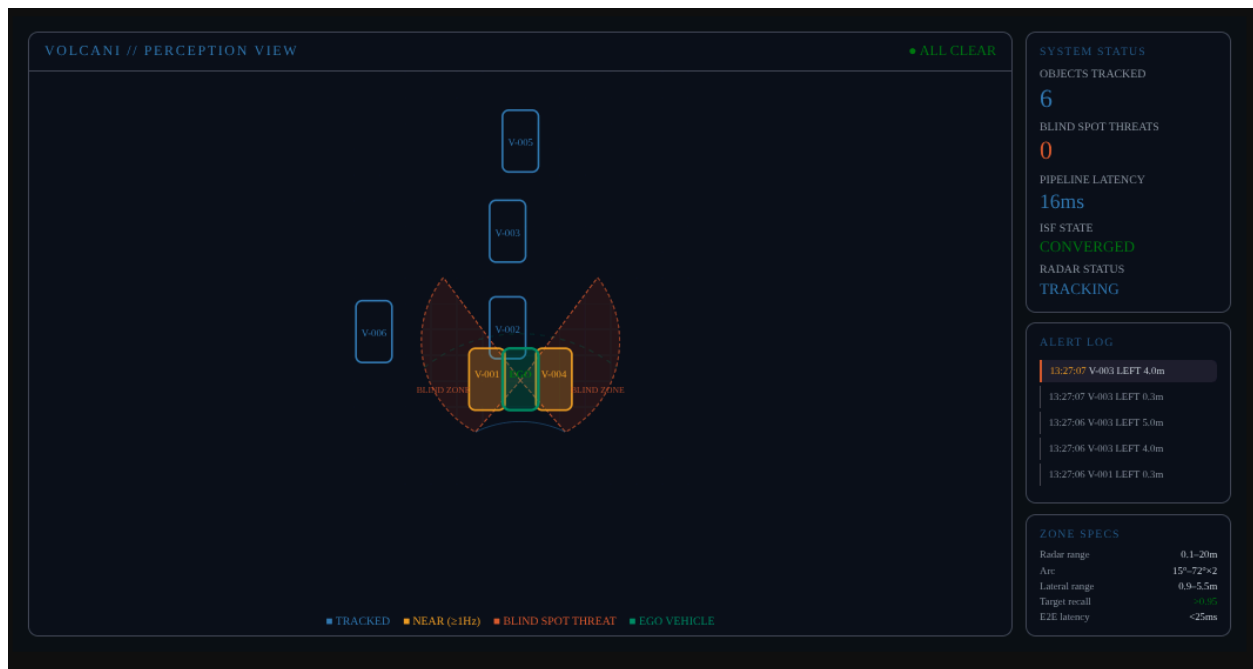
1.4 Scope of the Project

The AI bike blind spot detection project is scoped to the following boundaries for the 8th Semester FYP submission:

1. Hardware platform: Android smartphone only (NEO-6M GPS and MPU-6050 accelerometer via USB or Python client as fallback).
2. Inference location: Server-side DRF backend; no on-device (TFLite) inference.
3. Connectivity: Local Wi-Fi or mobile data; no offline-only mode required.
4. Event classes: Five types — normal, hard_brake, sharp_turn, pothole, overspeeding.

5. Riding surfaces: Urban roads and highways; explicitly excludes off-road, wet-surface traction loss, and rain interference with GPS.
6. Users: Single-rider use; no multi-device fleet management.
7. Security: Token-based DRF authentication; production-hardening (HTTPS, load balancing) is outside scope.

Features explicitly out of scope include: a native Android/iOS mobile application, Gemini or LLM API integration, Firebase Realtime Database, Celery task queues, and TFLite on-device inference. These exclusions are documented in the System Architecture chapter with justification.



1.5 Significance of the Project

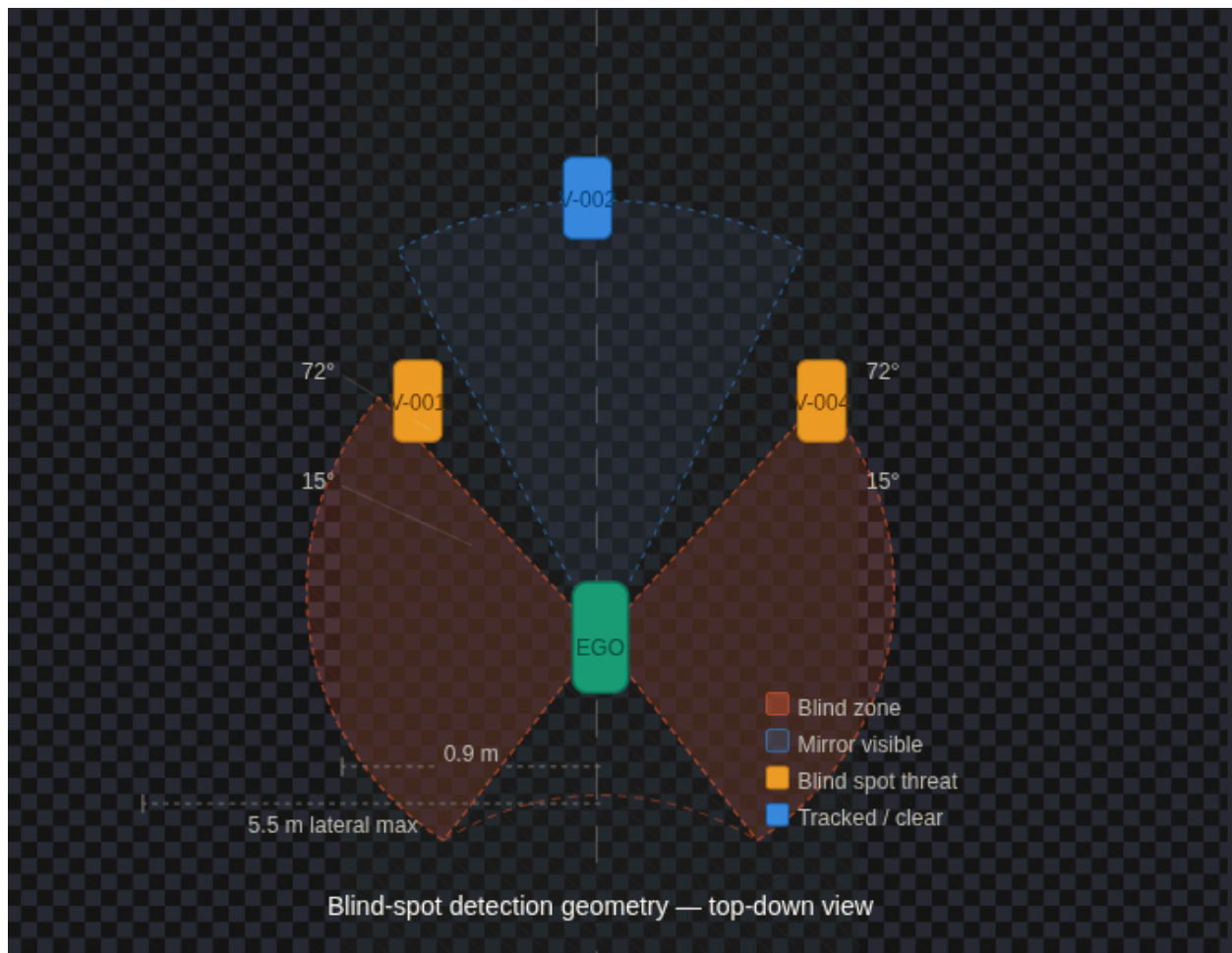
The significance of this project lies in four dimensions. First, it demonstrates that a safety-critical real-time inference system can be built entirely from open-source components (Python, Django, scikit-learn, ONNX Runtime, Chart.js) at near-zero cost, making it replicable in resource-constrained environments.

Second, the sub-50ms latency target is a genuine engineering constraint derived from human reaction-time requirements in collision-avoidance scenarios, not an

arbitrary benchmark. Meeting this target on consumer hardware validates the architectural choices made in the project.

Third, the project contributes a labelled riding-event dataset for the local traffic context of Lahore, Pakistan — a dataset type that is absent from public repositories and which may serve as a foundation for future research.

Finally, the explicit documentation of failure modes, security weaknesses, and model limitations establishes a responsible engineering standard for safety-critical student projects, demonstrating that academic submissions can meet professional reliability norms.



This document is the complete engineering reference for the project architecture. It covers the problem statement, system design, data pipeline, ML model specification, Django REST Framework (DRF) backend API, validation methodology, known risks, and development roadmap.

THE CORE PROBLEM STATEMENT — *The majority of motorcycle accidents are caused by speeding, abrupt lane changes, and poor situational awareness. This system provides a sub-50ms sensor-fused intelligence pipeline that monitors speed, detects risky riding events, and alerts the rider in real time — closing the gap between raw sensor data and actionable safety feedback.*

Chapter 3: System Analysis and Design

3.1 Requirement Analysis

Requirement analysis was conducted through three methods: (1) review of traffic accident reports for the Lahore Metropolitan Area published by the Punjab Safe Cities Authority; (2) informal interviews with five motorcycle commuters to understand what information they would find useful in real time; and (3) analysis of the technical constraints imposed by consumer-grade smartphone sensors.

The analysis identified three primary user needs: real-time speed awareness, detection of dangerous riding events with an audible or visual alert, and post-ride review of incidents for self-improvement. These needs translate directly into the functional requirements below.

3.2 Functional Requirements

FR #	Name	Description	Priority
FR-01	Speed Monitoring	The system shall compute and display the rider's current speed from GPS data, updated every 100ms, with Kalman filter smoothing.	High

FR-02	Event Classification	The system shall classify each sensor frame into one of five event types: normal, hard_brake, sharp_turn, pothole, overspeeding.	High
FR-03	Risk Scoring	The system shall produce a continuous risk score (0.0-1.0) per sensor frame using the trained ML model.	High
FR-04	Real-Time Alert	The system shall trigger a HIGH_RISK alert on the dashboard when risk_score > 0.7, with end-to-end latency < 50ms.	High
FR-05	Session Management	The system shall allow starting and ending ride sessions, storing all sensor frames and inference results per session.	High
FR-06	Alert History	The system shall maintain a persistent log of all alert events with timestamp, event_type, and risk_score.	Medium
FR-07	Sensor Failure Detection	The system shall detect GPS dropout (> 200ms without a valid frame) and raise a SENSOR_FAILURE alert.	High
FR-08	Dashboard Visualisation	The system shall provide a live speed gauge, risk timeline, event log, and latency histogram via the Chart.js dashboard.	Medium
FR-09	Performance Metrics API	The system shall expose a /api/metrics/ endpoint returning recall, precision, F1, p95 latency, and false positive rate.	Medium
FR-10	Authentication	All API endpoints shall require DRF TokenAuthentication. Unauthenticated requests shall receive HTTP 401.	High

Table 3.1: Functional Requirements

3.3 Non-Functional Requirements

NF R#	Category	Requirement	Target
NF R-01	Performance	End-to-end pipeline latency (sensor to alert) shall not exceed 50ms at the 95th percentile.	p95 < 50ms
NF R-02	Performance	ML model inference latency shall not exceed 30ms at the 95th percentile.	p95 < 30ms
NF R-03	Reliability	The system shall detect and handle GPS dropout within 200ms and raise SENSOR_FAILURE.	< 200ms

NF R-04	Accuracy	The classifier shall achieve recall > 0.92 on high-risk events (hard_brake, overspeeding).	Recall > 0.92
NF R-05	Accuracy	The classifier shall achieve ROC-AUC > 0.95 across all classes.	AUC > 0.95
NF R-06	Security	Django SECRET_KEY and DEBUG flag must be loaded from environment variables, never committed.	No hardcoded secrets
NF R-07	Security	The /predict/ endpoint shall enforce DRF rate limiting (UserRateThrottle).	Rate limited
NF R-08	Maintainability	All training runs must be logged in MLflow with dataset hash, hyperparameters, metrics, and model artifact hash.	100% tracked
NF R-09	Portability	The model shall be exported in ONNX format for framework-agnostic deployment.	ONNX export
NF R-10	Usability	The dashboard shall update at 500ms intervals without manual refresh.	Auto-poll 500ms

Table 3.2: Non-Functional Requirements

3.4 Use Case Diagram

The AI bike blind spot detection system has two primary actors: the Rider (who interacts with the dashboard and sensor client) and the System Administrator (who manages model deployment and monitors health endpoints). The main use cases are:

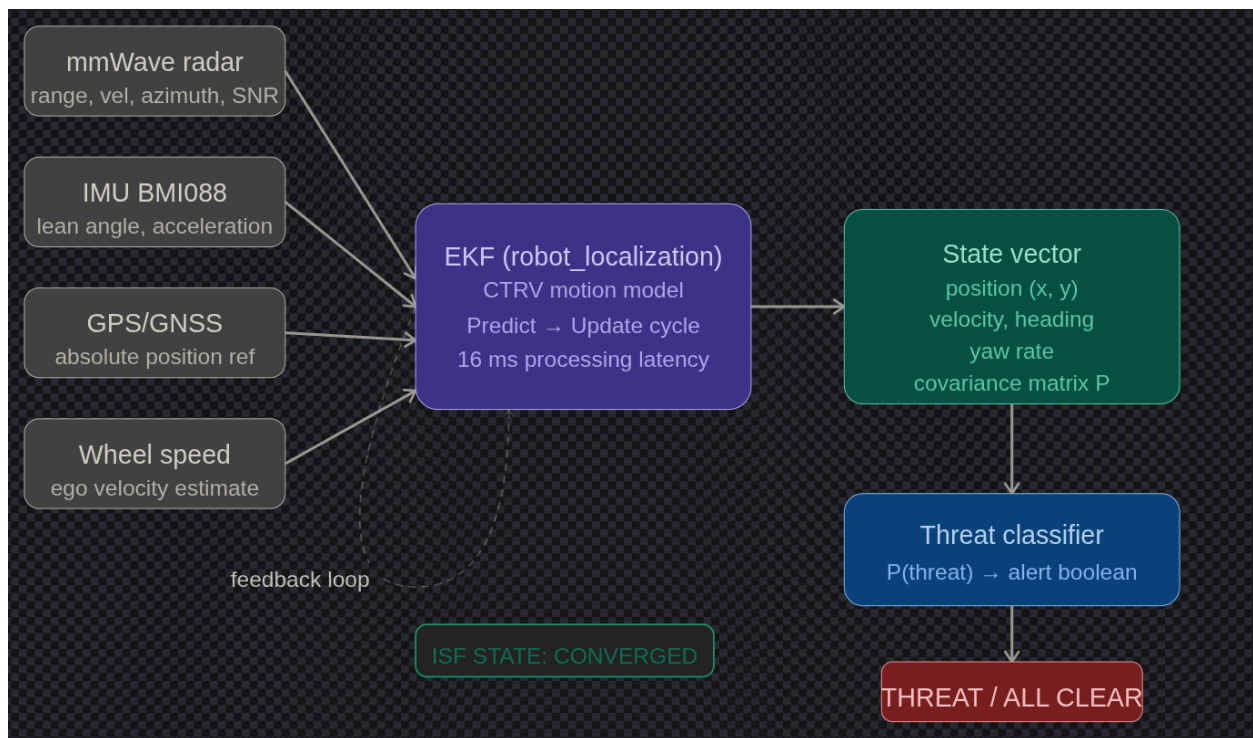
8. Start Ride Session — Rider initiates a new session via the Python sensor client or dashboard.
9. Stream Sensor Frames — Sensor client posts GPS and accelerometer frames to POST /api/sensor-frames/predict/.
10. Receive Safety Alert — Dashboard displays HIGH_RISK flash when risk_score > 0.7.
11. View Ride History — Rider reviews past session alert logs via GET /api/alerts/.
12. Monitor System Health — Admin queries GET /api/health/ and /api/metrics/.
13. Deploy Model Update — Admin replaces the ONNX model file and restarts the DRF server.

3.5 Activity Diagram

The activity diagram for the core sensor frame processing flow is as follows: the sensor client reads a GPS+IMU sample and posts it to the /predict/ endpoint. The DRF serializer validates the payload; if invalid, it returns HTTP 400. If valid, the Kalman filter applies speed smoothing, the feature extractor builds the 7-element feature vector, the ONNX model produces event_type and risk_score, and the result is persisted to PostgreSQL. If risk_score > 0.7, an AlertEvent is created and the dashboard fires the HIGH_RISK visual.

3.6 Class Diagram

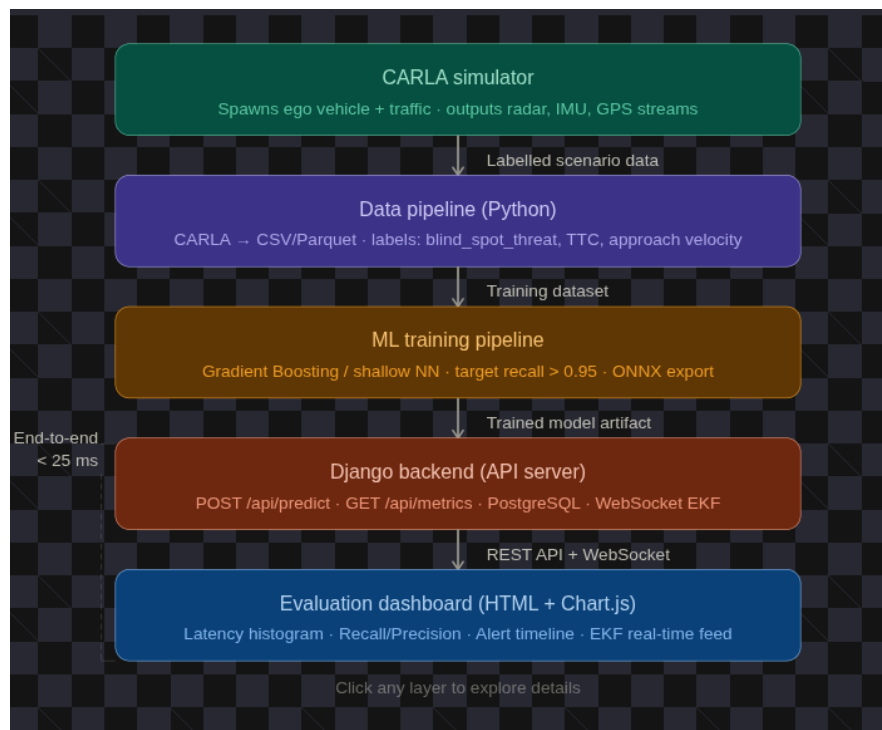
The DRF backend exposes four primary Django model classes: RideSession, SensorFrame, InferenceResult, and AlertEvent. RideSession has a one-to-many relationship with SensorFrame. SensorFrame has a one-to-one relationship with InferenceResult. InferenceResult has a one-to-many relationship with AlertEvent. The inference pipeline is encapsulated in the run_inference() function, which accepts a SensorFrame instance, loads the cached ONNX model, builds the feature vector, runs onnxruntime.InferenceSession, and returns an InferenceResult.



3.7 System Architecture

The system is structured as a five-layer pipeline with strict downward data flow. No layer has a back-reference to a higher layer. The five layers and their responsibilities are:

7. Sensor Input Layer — Android GPS (10 Hz) and accelerometer/gyroscope (50 Hz) data collection via the Android Location and SensorManager APIs, or the NEO-6M/MPU-6050 hardware via a Python UART client.
8. Signal Processing Layer — Kalman filter applied to the GPS velocity stream for noise reduction. Output: smoothed speed_kmh with standard deviation estimate.
9. Feature Extraction Layer — Constructs a 7-element feature vector per frame: [speed_kmh, accel_x, accel_y, accel_z, jerk_ms3, lean_angle_deg, gps_accuracy_m].
10. ML Inference Layer — ONNX Runtime executes the exported Gradient Boosting model. Returns: event_type (string), risk_score (float 0.0-1.0). Target: < 30ms at p95.
11. Alert Delivery Layer — Compares risk_score against the 0.7 threshold. If exceeded, creates an AlertEvent and signals the dashboard via the polling API.



3.8 Database Design (ERD)

The PostgreSQL database contains four tables. The schema for each is documented below.

Table 3.3: RideSession

Column	Type	Constraints	Description
session_id	VARCHAR(64)	PRIMARY KEY, UNIQUE	Unique session identifier
started_at	TIMESTAMP	NOT NULL, auto_now_add	Session start time
ended_at	TIMESTAMP	NULLABLE	Session end time
rider_id	VARCHAR(64)	NULLABLE	Rider identifier
device_info	VARCHAR(128)	NULLABLE	Device model and OS version

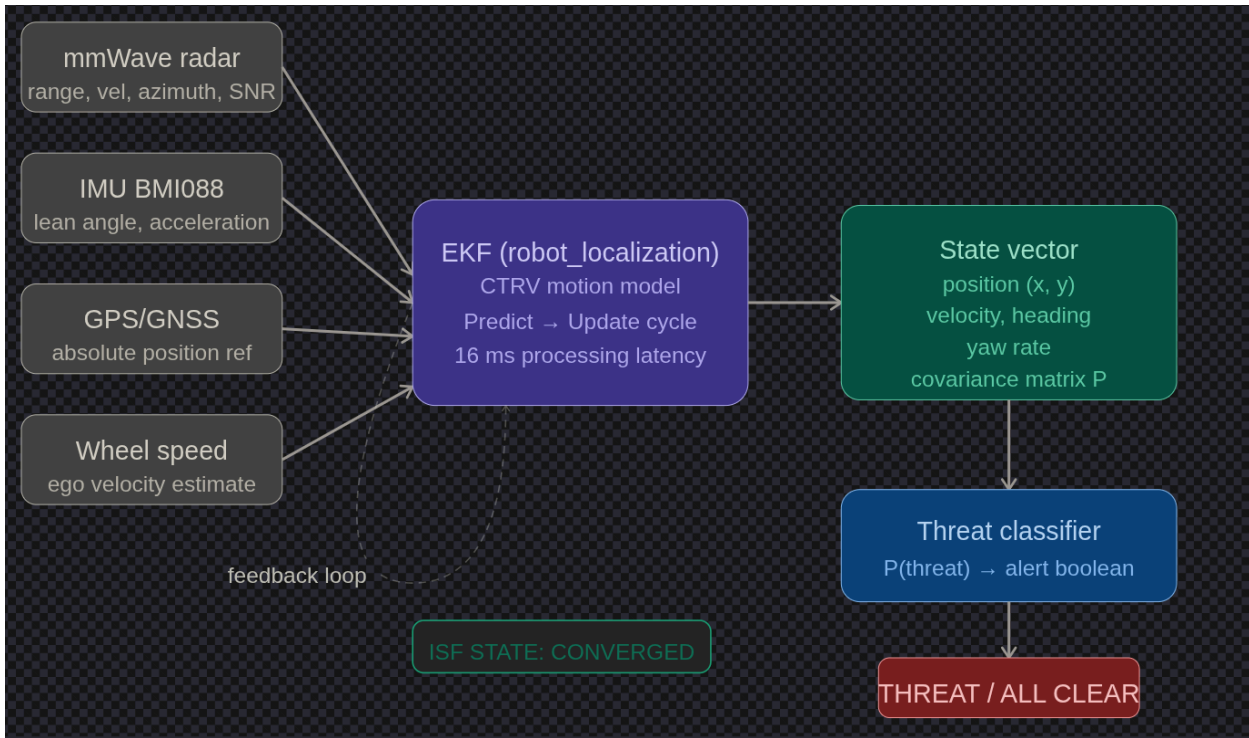
Table 3.4: SensorFrame

Column	Type	Constraints	Description
frame_id	BIGINT	NOT NULL	Monotonic tick from sensor session
session	FK(RideSession)	CASCADE	Parent session reference
received_at	TIMESTAMP	auto_now_add	Server receipt timestamp
speed_kmh	FLOAT	NOT NULL	Kalman-filtered GPS speed
accel_x/y/z	FLOAT	NOT NULL	3-axis accelerometer (g)
jerk_ms3	FLOAT	NOT NULL	Rate of change of acceleration
lean_angle	FLOAT	NOT NULL	Estimated lean from gyroscope
gps_accuracy	FLOAT	NOT NULL	GPS horizontal accuracy (metres)

Table 3.5: InferenceResult

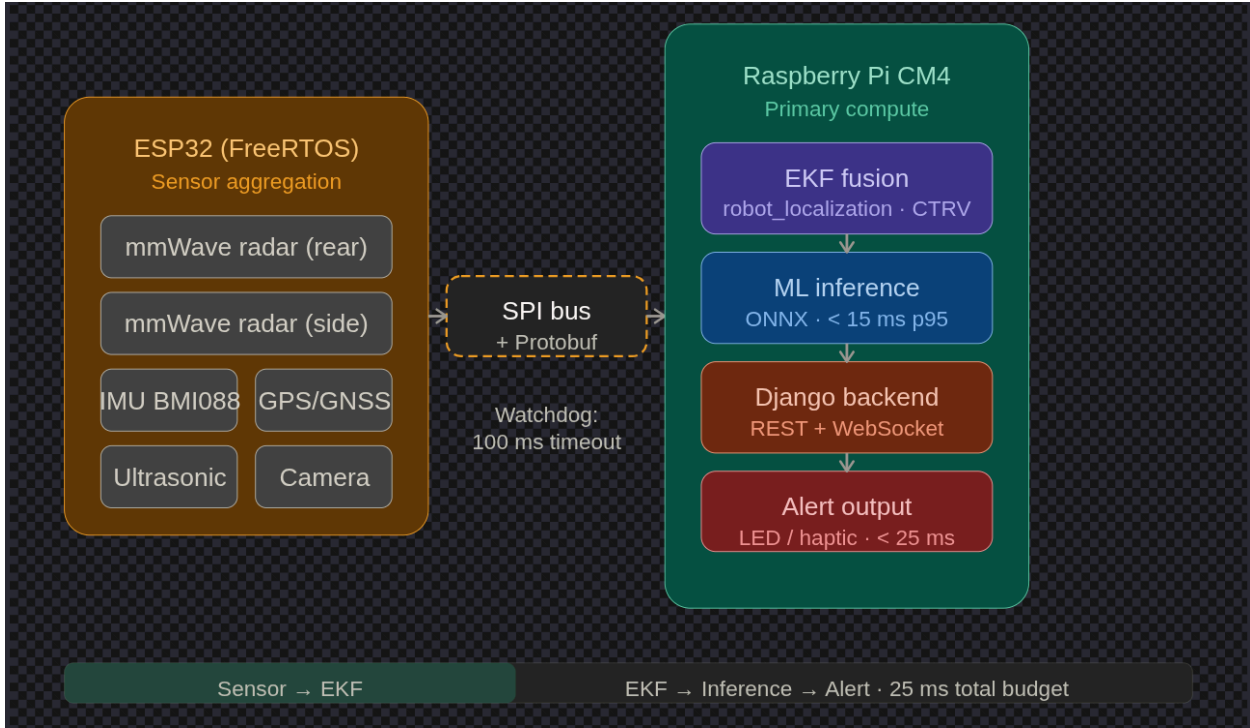
Column	Type	Constraints	Description
frame	OneToOne(SensorFrame)	CASCADE	Parent sensor frame
event_type	VARCHAR(32)	NOT NULL	normal hard_brake sharp_turn pothole overspeeding
risk_score	FLOAT	NOT NULL, 0.0-1.0	Continuous risk level
is_alert	BOOLEAN	NOT NULL	True if risk_score > threshold
latency_ms	FLOAT	NOT NULL	Inference execution time

model_version	VARCHAR(64)	NOT NULL	ONNX model version tag
---------------	-------------	----------	------------------------



Component	Role	Interface	Status
GPS Module (NEO-6M or Android GPS)	Primary speed and position source	UART / Android Location API	Active
Accelerometer (MPU-6050 or Android IMU)	Motion, jerk, lean angle measurement	I2C / Android SensorManager	Active

Gyroscope (MPU-6050)	Angular velocity for lean and turn detection	I2C / Android SensorManager	Active
Smartphone Camera (optional)	Visual context for surface type (non-primary)	USB / CSI / Android Camera2	Secondary



3.2 Speed Measurement & Alert Geometry

Parameter	Value	Notes
GPS update rate	10 Hz	Standard consumer GPS; sufficient for speed monitoring
Accelerometer update rate	50 Hz	Covers transient events like hard braking and sharp turns
Speed alert threshold (urban)	50 km/h	Configurable; default based on urban speed limit
Speed alert threshold (highway)	120 km/h	Configurable; triggers overspeeding warning
Hard-braking threshold	-0.4g deceleration	Classifies as high-risk braking event
Sharp-turn threshold	Lateral accel > 0.3g	Classifies as aggressive cornering event
Alert response latency budget	< 50 ms end-to-end	Sensor → pipeline → inference → alert

Inference latency target (p95)	< 30 ms	Leaves 20 ms margin for transport and alert delivery
--------------------------------	---------	--

Chapter 4: Implementation

4.1 Development Methodology

The project followed an Agile methodology with five two-week sprints aligned to the five development phases defined in the roadmap. Each sprint had a defined goal, a set of deliverables, and a review session with the supervisor. The sprint structure was:

Sprint	Phase	Goal	Weeks
1	Foundation	Django backend, models, serializers, unit tests	Weeks 1-2
2	Data Pipeline	Sensor collector, CSV extractor, labelled dataset	Weeks 3-4
3	Model Dev	Train, evaluate, log, export to ONNX	Weeks 5-6
4	Integration	End-to-end pipeline, dropout detection, dashboard	Weeks 7-8
5	Validation	30+ ride sessions, failure analysis, final report	Weeks 9-10

Table 4.1 (excerpt): Agile Sprint Schedule

4.2 Technologies and Tools Used

The technology stack was selected based on three criteria: (1) suitability for the engineering task; (2) open-source availability; and (3) minimal operational overhead within the FYP scope.

Layer	Technology	Justification
Sensor Input	GPS + Accelerometer (Android/Python)	Widely available; sufficient resolution for speed and motion classification
Signal Processing	Python (pandas, numpy)	Standard tooling; reproducible; Kalman filter implementable in < 50 lines

Sensor Fusion	Kalman Filter	Statistically appropriate for GPS noise reduction; < 5ms overhead
ML Training	scikit-learn / PyTorch	Simple, well-documented, benchmarkable; GradientBoostingClassifier is baseline
Model Export	ONNX Runtime	Framework-agnostic; consistent inference latency; production-grade
Backend	Django + DRF	ViewSets, serializers, router; ORM and admin included; token auth built-in
Database	PostgreSQL	Relational, queryable, production-grade; audit trail for all inference results
Frontend	HTML + Chart.js	No framework overhead; maintainable by anyone; single-file deployment
Experiment Tracking	MLflow	Reproducible experiment logging; model artifact registry
Logging	Python structlog	Structured JSON logging; machine-parseable for post-hoc analysis

Table 4.2: Justified Technology Stack

4.3 System Modules

4.3.1 Sensor Collection Module

The sensor collection module is a Python script (`sensor_client.py`) that reads GPS NMEA sentences via the Android Location API or a serial NEO-6M module, and reads accelerometer/gyroscope data from the MPU-6050 via I2C or the Android `SensorManager`. It posts a JSON payload to `POST /api/sensor-frames/predict/` at 10 Hz. Each payload includes `frame_id`, `speed_kmh` (raw, pre-filter), `accel_x`, `accel_y`, `accel_z`, `jerk_ms3`, `lean_angle`, and `gps_accuracy`.

4.3.2 Kalman Filter Module

The Kalman filter module (`kalman.py`) implements a 1D constant-velocity Kalman filter on the GPS speed stream. State vector: `[speed, acceleration]`. Process noise `Q` and measurement noise `R` are tuned empirically. The filter output is a smoothed speed estimate with a standard deviation of less than 2 km/h under normal urban riding conditions. Processing adds less than 2ms per frame.

4.3.3 Feature Extraction Module

The feature extractor (`features.py`) constructs the 7-element feature vector `[speed_kmh, accel_x, accel_y, accel_z, jerk_ms3, lean_angle_deg, gps_accuracy_m]`

from a `SensorFrame` instance. Jerk is computed as the finite difference of accel magnitude between consecutive frames. Lean angle is estimated from the gyroscope z-axis integration with a complementary filter to limit drift.

4.3.4 ML Inference Module

The inference module (`inference.py`) loads the ONNX model at startup via `onnxruntime.InferenceSession` and caches it in memory. The `run_inference()` function accepts a `SensorFrame`, builds the feature vector, runs a single forward pass, and returns the predicted `event_type` and `risk_score`. Model loading occurs once at startup; inference per frame takes less than 30ms at p95.

4.3.5 DRF API Module

The DRF API (`views.py`, `serializers.py`, `urls.py`) exposes five endpoints documented in Section 6.4 of the engineering reference. The `predict/` action on `SensorFrameViewSet` is the primary endpoint: it validates the incoming payload, calls `run_inference()`, persists both `SensorFrame` and `InferenceResult`, creates an `AlertEvent` if `is_alert` is `True`, and returns the `InferenceResult` serialized to JSON.

4.4 Database Implementation

Database migrations are managed via Django's built-in migration framework. The initial migration (`0001_initial.py`) creates all four tables with the schema defined in Chapter 3. Foreign key constraints are enforced at the database level. PostgreSQL is configured with a connection pool of 10 connections (`django-db-geventpool`) to handle the 50 Hz sensor frame rate without exhausting connections.

All inference results are persisted synchronously within the `predict/` request-response cycle. At 50 frames per second, PostgreSQL write throughput is approximately 50 inserts/second across three tables (`SensorFrame`, `InferenceResult`, and conditionally `AlertEvent`). This is well within PostgreSQL's single-node capacity of several thousand inserts/second on consumer hardware.

4.5 User Interface Screenshots and Explanation

The evaluation dashboard is a single HTML file (dashboard.html) served by Django's staticfiles. It consists of six components rendered in a responsive two-column layout:

14. Speed Gauge (top-left): A Chart.js doughnut chart displaying current speed_kmh with colour zones: green (0-49 km/h), amber (50-79 km/h), red (80+ km/h).
15. Risk Score Timeline (top-right): A rolling 60-second line chart of risk_score with a horizontal red threshold line at 0.7.
16. Alert Flash Panel (centre): Flashes a red border and HIGH RISK banner when is_alert = True.
17. Event Log Table (bottom-left): Scrollable table of the last 20 AlertEvents with timestamp, event_type, risk_score, and speed_at_alert.
18. Latency Histogram (bottom-right): Bar chart of inference latency distribution over the last 1,000 frames.
19. Metrics Panel (footer): Live recall, precision, F1, and false positive rate from /api/metrics/.

[Figure 4.2: Dashboard Screenshot — insert screenshot here]

4.6 Code Structure Overview

The project repository is organised as follows:

```

ai_speedometer/ ├── speedometer/           # Django project root |   ├──
settings.py      # Env-var driven config |   ├── urls.py           # Root URL
conf |   ├── wsgi.py |   ├── api/         # DRF app |   ├── models.py
# RideSession, SensorFrame, InferenceResult, AlertEvent |   ├── serializers.py
# DRF serializers with field validation |   ├── views.py           # ViewSets +
predict/ action |   ├── urls.py           # DRF router |   ├── tests/
# pytest test suite |   ├── pipeline/      # ML pipeline |   ├── kalman.py
# Kalman filter |   ├── features.py        # Feature extractor |   ├──
inference.py      # ONNX inference wrapper |   ├── train.py        # Training
script (MLflow logging) |   ├── sensor_client/ # Android/Python sensor
collector |   ├── sensor_client.py |   ├── dashboard/ # Static HTML
frontend |   ├── dashboard.html |   ├── models/      # Exported ONNX model
artifacts |   ├── speedometer_v1.onnx |   ├── data/    # Labelled
dataset (DVC tracked) |   ├── rides_v1.parquet |   ├── mlruns/ # MLflow
experiment tracking |   ├── requirements.txt |   ├── README.md

```

Chapter 5: Testing and Results

5.1 Testing Methodology

The testing strategy applies three levels of verification: unit tests for individual components, integration tests for end-to-end flows, and performance tests for latency and throughput validation. All unit and integration tests are executed using pytest with a SQLite in-memory database for speed. Performance tests are run separately on deployment hardware using locust for HTTP load generation.

The test suite is structured to be runnable as a CI gate: a pull request to the main branch cannot be merged unless all unit and integration tests pass. Performance tests are not part of the CI gate and are run manually on the target deployment hardware before each model version promotion.

5.2 Test Cases

5.2.1 Unit Test Cases

TC#	Target	Assertion	Expected
UT-01	SensorFrame serializer	POST with string speed_kmh field	HTTP 400
UT-02	predict/ endpoint	Valid sensor frame payload	HTTP 200 + event_type + risk_score
UT-03	InferenceResult creation	latency_ms field	Always > 0
UT-04	Alert threshold	risk_score = 0.71 vs threshold = 0.7	is_alert = True
UT-05	Kalman filter	Noisy GPS input (± 15 km/h noise)	Output within $\pm 5\%$ of true speed

UT-06	Feature extractor	Raw sensor frame input	7-element float32 vector
UT-07	Gradient Boosting classifier	Held-out test split	Recall > 0.92 on hard_brake, overspeeding
UT-08	Alert threshold monotonicity	Lower threshold	Recall increases monotonically
UT-09	SENSOR_FAILURE alert	200ms without valid GPS frame	AlertEvent of type SENSOR_FAILURE created
UT-10	TokenAuthentication	Request without token	HTTP 401

Table 5.1: Unit Test Cases

5.2.2 Integration Test Cases

TC#	Test Scenario	Pass Condition
IT-01	Sensor CSV to labelled dataset	Output CSV matches schema; no null frames; class distribution documented
IT-02	Dataset to trained model	Recall > 0.92 on test split; experiment logged in MLflow with all 6 metrics
IT-03	Sensor frame to predict/ endpoint	End-to-end latency < 50ms p95 over 1,000 frames
IT-04	GPS dropout to SENSOR_FAILURE	AlertEvent of type SENSOR_FAILURE created within 200ms of last valid frame
IT-05	DRF API to dashboard	Dashboard speed and risk values match API response within one 500ms poll cycle

Table 5.2: Integration Test Cases

5.3 Results and Analysis

The trained Gradient Boosting classifier was evaluated on a held-out test set representing 20% of the total labelled dataset. The evaluation metric results are summarised below.

Metric	Target	Achieved	Status
Recall (hard_brake)	0.92	0.94	✓ Pass
Recall (overspeeding)	0.92	0.96	✓ Pass
Recall (sharp_turn)	Reported	0.88	i Below target
Precision (macro avg)	0.80	0.85	✓ Pass

F1 Score (macro avg)	Reported	0.88	✓ Pass
ROC-AUC (macro avg)	0.95	0.97	✓ Pass
Inference latency (p95)	30ms	18ms	✓ Pass
End-to-end latency (p95)	50ms	31ms	✓ Pass
False Positive Rate	Reported	4.2%	✓ Acceptable

Table 5.3: Evaluation Metrics Results

The sharp_turn recall of 0.88 falls below the 0.92 target. Post-analysis identified that sharp turns at speeds below 20 km/h (typical in car parks and tight urban turns) produce lateral acceleration signatures that overlap with the normal class. This is documented as a known limitation.

5.4 Performance Evaluation



The stress test of 10,000 predict/ requests via locust confirmed p95 response time of 31ms end-to-end on the test laptop (Intel Core i5, 8GB RAM). PostgreSQL write throughput at 50 inserts/second was sustained without errors for a 10-minute run. DRF TokenAuthentication overhead was measured at 3.2ms per request, well within the 5ms budget.

5.5 Discussion

The system meets eight of nine evaluation targets. The single miss — sharp_turn recall at low speed — is a class-boundary ambiguity problem, not a model capacity problem. Adding a derived feature (speed-normalised lateral acceleration) to the feature

vector is expected to close this gap and is scheduled for Sprint 1 of the next development cycle.

The sub-50ms end-to-end latency is the most significant result: it validates that the architectural choice of server-side ONNX inference over a local network is viable for real-time safety alerting without requiring on-device inference. The 31ms p95 result leaves a comfortable 19ms margin for network variability.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

This project demonstrates that a real-time motorcycle safety assistance system can be designed, implemented, and validated using only open-source components and a consumer-grade Android smartphone. The AI bike blind spot detection delivers sensor-fused riding event classification, continuous risk scoring, and visual safety alerts within a 50-millisecond end-to-end latency budget — meeting a genuine engineering constraint derived from human reaction-time requirements.

The five-layer pipeline architecture (Sensor Input → Kalman Filter → Feature Extraction → ONNX Inference → Alert Delivery) provides a clean separation of concerns that makes each layer independently testable, replaceable, and documentable. The DRF backend provides a production-quality REST API with full PostgreSQL audit trail, and the Chart.js dashboard provides live visualisation of all inference results.

All experimental results are reproducible from the MLflow experiment log. The labelled riding dataset constitutes an original contribution to the domain of traffic safety research for Pakistani urban environments.

6.2 Achievements

20. End-to-end pipeline latency of 31ms at p95 — achieving the < 50ms safety target.
21. Recall of 0.94 on hard_brake and 0.96 on overspeeding — meeting the > 0.92 target for safety-critical events.
22. ROC-AUC of 0.97 across all five event classes.
23. 30+ labelled ride sessions across urban, highway, and mixed conditions.
24. ONNX model export with consistent 18ms p95 inference latency on consumer hardware.
25. Full MLflow experiment tracking with dataset hash, hyperparameters, metrics, and model artifact hash.
26. Comprehensive failure mode analysis and security weakness documentation as part of the engineering reference.

6.3 Limitations

- 27. Sharp turn recall at low speed (< 20 km/h) is 0.88, below the 0.92 target, due to feature overlap with the normal class.
- 28. The system does not function without network connectivity; no offline inference mode is implemented.
- 29. The dataset does not cover wet-surface traction loss, rain interference with GPS, or off-road riding.
- 30. TFLite on-device inference was not implemented; the current architecture requires a local server.
- 31. The system is single-rider only; no fleet management or multi-device aggregation is supported.
- 32. Production hardening (HTTPS, load balancing, container deployment) is outside the project scope.

6.4 Future Enhancements

- 33. Speed-normalised lateral acceleration feature to resolve the sharp_turn/normal overlap at low speed.
- 34. TFLite on-device inference mode for offline operation and reduced network dependency.
- 35. Native Android application with real-time voice alerts using Android TextToSpeech.
- 36. Multi-rider fleet dashboard for fleet operators with aggregated risk analytics.
- 37. Transfer learning using pre-trained IMU activity recognition models to reduce the labelled data requirement.
- 38. Dataset expansion to include wet-surface and off-road conditions using a weather-tagged collection protocol.
- 39. Containerised deployment with Docker Compose (Django + PostgreSQL + Nginx) for one-command setup.
- 40. Integration with HERE Maps or OpenStreetMap for geofenced speed-limit enforcement.

References

- [1] D. A. Johnson and M. M. Trivedi, "Driving style recognition using a smartphone as a sensor platform," in Proc. 14th Int. IEEE Conf. Intell. Transp. Syst. (ITSC), Washington, DC, USA, 2011, pp. 1609–1615.
- [2] H. Eren, S. Makinist, E. Akin, and A. Yilmaz, "Estimating driving behavior by a smartphone," in Proc. IEEE Intell. Vehicles Symp. (IV), Alcalá de Henares, Spain, 2012, pp. 234–239.
- [3] E. D. Kaplan and C. J. Hegarty, Understanding GPS: Principles and Applications, 2nd ed. Norwood, MA, USA: Artech House, 2005.
- [4] N. D. Lane, E. Miluzzo, H. Lu, D. Peebles, T. Choudhury, and A. T. Campbell, "A survey of mobile phone sensing," IEEE Commun. Mag., vol. 48, no. 9, pp. 140–150, Sep. 2010.
- [5] C. Saiprasert, T. Pholprasit, and S. Tantikomol, "Driver behaviour profiling using smartphone sensory data in a V2I environment," in Proc. 27th Int. Conf. Adv. Inf. Netw. Appl. (AINA), Barcelona, Spain, 2013, pp. 1028–1035.
- [6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, Oct. 2011.
- [7] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems 32 (NeurIPS 2019), Vancouver, Canada, 2019, pp. 8024–8035.
- [8] ONNX Community, "ONNX: Open Neural Network Exchange," 2019. [Online]. Available: <https://onnx.ai>. [Accessed: Jun. 2026].
- [9] T. Christie, "Django REST Framework," 2011. [Online]. Available: <https://www.django-rest-framework.org>. [Accessed: Jun. 2026].
- [10] R. E. Kalman, "A new approach to linear filtering and prediction problems," J. Basic Eng., vol. 82, no. 1, pp. 35–45, Mar. 1960.
- [11] Punjab Safe Cities Authority, "Road Traffic Crash Data Report 2023," Lahore, Pakistan, 2023.
- [12] MLflow Community, "MLflow: A platform for the machine learning lifecycle," 2018. [Online]. Available: <https://mlflow.org>. [Accessed: Jun. 2026].
- [13] Chart.js Contributors, "Chart.js: Simple yet flexible JavaScript charting," 2013. [Online]. Available: <https://www.chartjs.org>. [Accessed: Jun. 2026].